

Practical – 4

4) Write a C program to create multiple child processes using `fork()` and synchronize their execution using `wait()` and `waitpid()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid1, pid2;
    int status;

    pid1 = fork();

    if (pid1 == 0)
    {
        printf("\nChild 1 Process\n");
        printf("Child 1 PID = %d\n", getpid());
        exit(0);
    }

    pid2 = fork();

    if (pid2 == 0)
    {
        printf("\nChild 2 Process\n");
        printf("Child 2 PID = %d\n", getpid());
        exit(0);
    }

    printf("\nParent Process\n");
    printf("Parent PID = %d\n", getpid());

    wait(&status);
    printf("\nFirst Child Completed\n");

    waitpid(pid2, &status, 0);
    printf("Second Child Completed\n");

    printf("\nAll Child Processes Completed\n");

    return 0;
}
```

```
gowtham@DESKTOP-2F03K3T:~$ cd 2520090202_oss
gowtham@DESKTOP-2F03K3T:~/2520090202_oss$ mkdir practical
gowtham@DESKTOP-2F03K3T:~/2520090202_oss$ cd practical
gowtham@DESKTOP-2F03K3T:~/2520090202_oss/practical$ nano practical4.c
gowtham@DESKTOP-2F03K3T:~/2520090202_oss/practical$ gcc practical4.c -o practical4
gowtham@DESKTOP-2F03K3T:~/2520090202_oss/practical$ ./practical4
```

Parent Process

Parent PID = 10345

Child 1

Child 1 PID = 10346

First Child Completed

Child 2

Child 2 PID = 10347

Second Child Completed

All Child Processes Completed

```
gowtham@DESKTOP-2F03K3T:~/2520090202_oss/practical$ █
```